

Optimisation de l'Infrastructure Technique pour Jean (CTO)

Information importante avant de démarrer.

Il est interdit de VIBE CODER, nous recherchons avant tout des développeurs pour rejoindre notre équipe.

Contexte:

Dans ce test, vous devez développer une application modulaire répondant à deux problématiques concrètes pour une PME française, en structurant votre solution en plusieurs étapes (ou nœuds) qui se succèdent pour transformer et enrichir les données.

Vous êtes libre de choisir le langage, le framework et les outils (par exemple, langgraph, LangChain, etc.), ainsi que les modèles de LLM (OpenAi, Mistral, Claude, etc.).

1. Optimisation de l'Infrastructure Technique pour Jean (CTO)

Objectif

- Développer une solution qui permet de :
 1. **Ingestion et Analyse de Données Techniques :**
 - § Traiter des données simulées issues d'un fichier de log en JSON.
 2. **Détection d'Anomalies :**
 - § Identifier des indicateurs anormaux (par exemple, une utilisation excessive du CPU, une latence élevée, etc.).
 3. **Génération de Recommandations :**
 - § Produire un rapport structuré (format JSON) proposant des actions concrètes pour optimiser la performance de l'infrastructure (par exemple, répartition de charge, ajustement des ressources, etc.).

Exigences Techniques

Format d'Entrée Exemple

```
{  
  "timestamp": "2023-10-01T12:00:00Z",  
  "cpu_usage": 85,  
  "memory_usage": 70,  
  "latency_ms": 250,  
  "disk_usage": 65,  
  "network_in_kbps": 1200,  
  "network_out_kbps": 900,  
  "io_wait": 5,  
  "thread_count": 150,  
  "active_connections": 45,  
  "error_rate": 0.02,  
  "uptime_seconds": 360000,  
  "temperature_celsius": 65,  
  "power_consumption_watts": 250,  
  "service_status": {  
    "database": "online",  
    "api_gateway": "degraded",  
    "cache": "online"  
  }  
}
```

2. Architecture Multi-Nœuds

Votre solution doit être organisée en plusieurs étapes (par exemple : ingestion → analyse → recommandation). La structuration de ces étapes est laissée à votre discrétion.

3. Documentation

Expliquez vos choix techniques et architecturaux (langage, bibliothèques, etc.) via des commentaires dans le code ou un document annexe.

Format de Sortie Attendu

Votre application doit générer un JSON conforme au schéma générique suivant :

```
{
  "timestamp": "string (ISO 8601)",
  "insights": {
    "average_latency_ms": "number",
    "max_cpu_usage": "number",
    "max_memory_usage": "number",
    "error_rate": "number",
    "uptime_seconds": "number"
  },
  "anomalies": [
    {
      "metric": "string",
      "value": "number",
      "threshold": "number",
      "severity": "string (low|medium|high)",
      "description": "string"
    }
  ],
  "recommendations": [
    {
      "id": "string",
      "action": "string",
      "target": "string",
      "parameters": "object",
      "benefit_estimate": "string"
    }
  ],
  "service_status_summary": {
    "online": ["string"],
    "degraded": ["string"],
    "offline": ["string"]
  }
}
```

Détails des types :

- **timestamp** : chaîne de caractères ISO 8601.
- **insights** : objet contenant des nombres.
- **anomalies** : tableau d'objets décrivant chaque anomalie.
- **recommendations** : tableau d'objets pour les actions à mener.
- **service_status_summary** : objet mappant chaque statut à un tableau de chaînes.

Consigne de livraison

Merci de nous transmettre les éléments suivants par email :

- Un fichier output.json contenant le JSON attendu.
- Un fichier gitingest.txt (<https://gitingest.com/>), en excluant les dépendances (par exemple : node_modules, etc.).

Dès réception, l'un de nos leads techniques effectuera une revue de votre code et décidera des étapes suivantes du processus de recrutement.

En cas de retour positif, nous fixerons un rendez-vous dans nos locaux (73 rue anatole france 92300 levallois-perret) pour un entretien final d'une heure, au cours duquel vous aurez 30 minutes pour ajouter une fonctionnalité à ce projet.

Merci et bon test !